

MANUEL D'APPRENTISSAGE PAR LA PRATIQUE

SPRINT 3 — TÂCHE 02 : CONTRÔLE D'ACCÈS

AUTOMATIQUE PAR MODULE

Guide d'ingénierie active pour programmer de vos propres mains le moteur de vérification automatique des permissions du profil sur chaque route de l'API DRF (Socle RBAC, Invariants HTTP, Zéro Fuite).

Sprint : Sprint 3 (Gestion de Projets & RBAC)	Périmètre : apps/core & config/settings
Rôle Développeur : Lead Backend Django / Architecte DRF	Niveau de Compétence : <i>Homo Docens</i>
Architecture : Multi-Tenant DRF • Sécurité par Défaut	Contraintes : Zéro Copier-Coller • Saisie Manuelle

Cadrage Stratégique SCQA (Minto Pyramid)

Situation : La Tâche 01 a établi le modèle de données des rôles et des habilitations par module (Role, RoleModulePermission, ProjetRoleModuleOverride) pour les 12 modules BTP.

Complication : La vérification des droits dans l'API reste manuelle et éclatée vue par vue. Si un développeur oublie d'associer PermissionModule.pour(...) sur une nouvelle vue, n'importe quel utilisateur connecté (même un magasinier ou un visiteur) peut exécuter des actions critiques sur des modules non autorisés (ex: créer un projet, approuver un budget). La sécurité par omission est inacceptable.

Question : Comment bâtir une classe de permission DRF universelle et automatique, activée globalement, qui détecte le module de chaque route, détermine le niveau d'accès requis selon la méthode HTTP ou l'action, valide le profil en mémoire sans multiplier les requêtes SQL (problème N+1), tout en gérant les exemptions et surcharges ?

Orientation : Ce manuel vous guide pas-à-pas pour concevoir AutoModulePermission, les décorateurs @exempt_module_permission et @exige_niveau, et intégrer le contrôle au cœur de DRF.

Neuro-Pédagogie Active : Saisie Manuelle Obligatoire

Règle Absolue d'Apprentissage Actif (Stanislas Dehaene & John Sweller) :

L'IA ne doit jamais taper ce code à votre place. La lecture passive crée un biais d'illusion de compétence. En saisissant vous-même chaque méthode d'inspection, chaque bloc d'exception et chaque test, vos neurones moteurs ancrent la mécanique interne de Django REST Framework jusqu'à l'automatisation subconsciente parfaite.

SECTION 1 : FILM MENTAL & OBJECTIF SACRÉ (JOSEPH MURPHY)

1.1 Visualisation Subconsciente de la Forteresse Numérique (Murphy)

Avant de toucher au clavier, fermez les yeux et installez-vous dans un état de calme profond. Visualisez l'architecture de votre API comme une forteresse médiévale dotée d'une herse intelligente. Chaque requête HTTP (GET, POST, PATCH, DELETE) qui franchit le pont-levis est immédiatement inspectée. En moins d'une milliseconde, le système sait quel quartier de la forteresse (Module Projets, Chantier, Finance, Achats) la requête vise. Il consulte instantanément le profil du voyageur. Si le voyageur est un simple observateur et qu'il tente de franchir la salle des coffres (POST sur Finance), la herse s'abat instantanément avec un code **HTTP 403 Forbidden** limpide et net. Aucune donnée n'a fuité, aucun calcul inutile n'a été fait. Le Directeur Général et le Fondateur, quant à eux, traversent chaque porte avec une fluidité parfaite. Sentez la fierté et la sérénité que procure un système sécurisé par conception.

Film Mental : Visualisation du Système Accompli

Affirmation d'Ancrage Subconscient :

« Mon API est une forteresse inviolable et élégante. Chaque route est protégée par défaut. La sécurité découle de la simplicité et de l'harmonie de mon code. Mon esprit enregistre avec clarté chaque règle et chaque invariant. »

1.2 La Loi de l'Effort Inversé appliquée à la Sécurité Logicielle

Le Dr. Joseph Murphy enseigne la **Loi de l'Effort Inversé** : forcer la volonté engendre la résistance. En sécurité logicielle, essayer de se souvenir de protéger chaque vue manuellement (vue par vue, action par action) est voué à l'échec à cause de la fatigue cognitive et du turnover d'équipe. La solution élégante et souveraine consiste à **inverser l'effort** : faire de la sécurité l'état par défaut (*Secure by Default*). Si vous ne faites rien, la route est protégée. Pour l'ouvrir, il faut un geste conscient et explicite (ex: `@exempt_module_permission`). Ainsi, l'oubli humain protège au lieu d'exposer.

SECTION 2 : ARCHITECTURE & INVARIANTS (PÓLYA / DEHAENE)

2.1 Le Cycle d'Exécution d'une Requête dans Django REST Framework

Pour maîtriser le contrôle d'accès automatique, vous devez décomposer mentalement le cycle de vie précis d'une requête HTTP lorsqu'elle atteint Django REST Framework :

- **1. Middleware HTTP** : Résolution du tenant (schéma PostgreSQL) et validation CORS / IP.
- **2. Résolution d'URL (Router)** : Identification du pattern d'URL et sélection de la vue DRF cible.
- **3. Dispatch de la Vue (APIView.dispatch)** : Création de l'objet `rest_framework.request.Request`.
- **4. Méthode initial()** : Déclenchement séquentiel de l'authentification (`perform_authentication`), puis vérification des permissions globales (`check_permissions`), puis limitation de débit (`throttling`).
- **5. Évaluation de `check_permissions(request)`** : Itération sur chaque classe de `permission_classes`. Si une seule méthode `has_permission()` renvoie `False`, une exception `PermissionDenied (403)` est levée.
- **6. Exécution de la méthode handler** : `get()`, `post()`, etc., ou appel à `check_object_permissions()`.

2.2 Algorithme de Résolution Triangulaire du Module

Comment la permission automatique sait-elle quel module correspond à la requête ? Elle applique une heuristique de résolution en entonnoir (Pólya - Invariant de Spécification) :

Ordre de Priorité	Mécanisme de Détection	Exemple Concret	Cas d'Usage
1. Explicite (Vue)	Attribut <code>permission_module</code> ou <code>module_name</code>	<code>module_name = ModuleChoix.CHANTIER</code>	Surcharge manuelle sur une vue atypique
2. Automatique (App)	Inspection du package Python de la vue (<code>view.__module__</code>)	<code>apps.projets.views</code> → <code>projets</code>	Couvre 95% des endpoints standards
3. Automatique (URL)	Inspection du chemin d'URL (<code>request.path</code>)	<code>/api/v1/finance/...</code> → <code>finance</code>	Filet de sécurité pour les vues partagées

2.3 Déduction du Niveau d'Accès Requis par Méthode HTTP

Le système associe automatiquement la méthode HTTP au niveau d'accès minimal requis :

Méthode HTTP / Action	Niveau d'Accès Requis	Valeur Numérique	Règle Métier Associée
GET, HEAD, OPTIONS	NiveauAcces.LECTURE	1	Consultation en lecture seule sans altération d'état.
POST, PUT, PATCH, DELETE	NiveauAcces.ECRITURE	2	Création, mise à jour ou suppression de ressources.
Action décorée <code>@exige_niveau(VALIDATION)</code>	NiveauAcces.VALIDATION	3	Signature de bon, clôture de situation, validation de PV.

2.4 Invariants de Sécurité Inaliénables & Performance O(1)

Les Invariants Fondamentaux du Système

- **1. Le Bypass Souverain** : Les Super Admin, le Propriétaire du compte (`is_owner=True`), le Directeur Général (`is_dg=True`) et les rôles système AD ou DG possèdent l'accès absolu et immédiat sans aucune requête SQL additionnelle.
- **2. Le Cache de Requête (Request-Scoped Memoization)** : Pour éliminer le problème des requêtes SQL N+1, la matrice des permissions du rôle de l'utilisateur est chargée une seule fois par cycle de requête et mise en cache directement sur l'instance `request._module_perms_cache`.
- **3. Les Exemptions Strictes** : Certaines routes transverses ne sont rattachées à aucun des 12 modules métier (auth, inscription, profil connecté /me/, Swagger, sondes de santé). Elles doivent être expressément exemptées.

SECTION 3 : GUIDE D'IMPLÉMENTATION PAS-À-PAS (POUR VOS MAINS)

3.1 Étape 1 : Création du Module de Sécurité Transverse

Vous allez créer le fichier `apps/core/permissions_module.py`. Il regroupe les décorateurs d'exemption, la fonction de résolution de module et la classe de permission universelle `AutoModulePermission`. Tapez chaque ligne avec attention :

Fichier à éditer / créer : `apps/core/permissions_module.py` (Partie 1 : Constantes & Décorateurs)

```
# apps/core/permissions_module.py
"""Contrôle d'accès automatique par module pour Django REST Framework.

Résout automatiquement le module et le niveau requis (LECTURE, ECRITURE, VALIDATION)
sur chaque route de l'API sans intervention manuelle répétée.
"""

import re
from functools import wraps
from rest_framework import permissions
from rest_framework.exceptions import PermissionDenied
from apps.core.enums import ModuleChoix, NiveauAcces, RoleGlobal

# Clés de métadonnées pour les vues et handlers
ATTR_EXEMPT = "_module_permission_exempt"
ATTR_MODULE = "permission_module"
ATTR_NIVEAU = "_module_permission_niveau"

# Modules connus du système BTP
MODULES_VALIDES = set(ModuleChoix.values)

# Routes par défaut exemptées de la vérification par module
PREFIXES_EXEMPTES = (
    "/api/v1/auth/",
    "/api/v1/inscription/",
    "/api/v1/utilisateurs/me/",
    "/api/v1/schema/",
    "/api/v1/docs/",
    "/api/health/",
    "/api/v1/maintenance/",
)

def exempt_module_permission(view_or_func):
    """Décorateur indiquant qu'une vue ou un endpoint est exempté du contrôle par module."""
    setattr(view_or_func, ATTR_EXEMPT, True)
    return view_or_func

def exige_niveau(niveau: int):
    """Décorateur fixant explicitement le niveau d'accès requis pour une action donnée."""
    def decorator(view_or_func):
        setattr(view_or_func, ATTR_NIVEAU, int(niveau))
        return view_or_func
    return decorator
```

3.2 Étape 2 : Fonctions d'Inspection et de Cache

Ajoutez ensuite les fonctions de résolution de module et de mise en cache mémoire :

Fichier à éditer / créer : apps/core/permissions_module.py (Partie 2 : Résolution & Cache)

```
def resoudre_module_requete(view, request) -> str | None:
    """Résout le nom du module applicatif par inspection de la vue, du package ou de l'URL."""
    # 1. Attribut explicite sur la vue (ex: module_name ou permission_module)
    for attr in (ATTR_MODULE, "module_name", "module"):
        val = getattr(view, attr, None)
        if val and str(val) in MODULES_VALIDES:
            return str(val)

    # 2. Inspection du nom du package Python de la vue (ex: apps.chantier.views -> chantier)
    module_path = getattr(view, "__module__", "")
    segments = module_path.split(".")
    if len(segments) >= 2 and segments[0] == "apps":
        app_name = segments[1]
        if app_name in MODULES_VALIDES:
            return app_name

    # 3. Inspection du préfixe d'URL (ex: /api/v1/projets/...)
    path = request.path_info or request.path or ""
    match = re.match(r"^/api/v1/([a-zA-Z0-9_-]+)/", path)
    if match:
        segment = match.group(1).replace("-", "_")
        if segment in MODULES_VALIDES:
            return segment

    return None

def get_permissions_utilisateur_tenant(utilisateur, request=None) -> dict[str, int]:
    """Récupère et met en cache la matrice {module: niveau} pour l'utilisateur courant."""
    if request is not None and hasattr(request, "_module_perms_cache"):
        return request._module_perms_cache

    from django.apps import apps as registre
    Role = registre.get_model("accounts", "Role")
    RoleModulePermission = registre.get_model("accounts", "RoleModulePermission")

    role = getattr(utilisateur, "role_personnalise", None)
    if not role and getattr(utilisateur, "role_global", None):
        role = Role.objects.filter(
            code=utilisateur.role_global, supprime_le__isnull=True
        ).first()

    if not role:
        perms_dict = {}
    else:
        perms_qs = RoleModulePermission.objects.filter(
            role=role, supprime_le__isnull=True
        ).values_list("module", "niveau")
        perms_dict = {mod: niv for mod, niv in perms_qs}

    if request is not None:
        request._module_perms_cache = perms_dict

    return perms_dict
```

3.3 Étape 3 : La Classe Principale AutoModulePermission

Voici le cœur du moteur : AutoModulePermission. Elle s'intègre directement dans DRF et orchestre toutes les vérifications de manière transparente :

Fichier à éditer / créer : apps/core/permissions_module.py (Partie 3 : AutoModulePermission)

```
class AutoModulePermission(permissions.BasePermission):
    """Vérification automatique et universelle des habilitations de module par profil."""

    message = "Vos habilitations ne vous permettent pas d'accéder à ce module."

    def has_permission(self, request, view) -> bool:
        user = request.user
        if not user or not user.is_authenticated:
            return False

        # 1. Bypass souverain : Super Admin, Owner, DG
        if (
            user.is_superuser
            or getattr(user, "is_owner", False)
            or getattr(user, "is_dg", False)
            or getattr(user, "role_global", None) in (RoleGlobal.ADMIN, RoleGlobal.DIRECTEUR_GENERAL)
        ):
            return True

        # 2. Vérification des exemptions (Attribut sur la vue ou préfixe d'URL)
        if getattr(view, ATTR_EXEMPT, False):
            return True

        path = request.path_info or request.path or ""
        for prefix in PREFIXES_EXEMPTES:
            if path.startswith(prefix):
                return True

        # 3. Résolution du module
        module = resoudre_module_requete(view, request)
        if not module:
            # Si aucun module n'est associé, la route est neutre (laisser passer aux autres permissions)
            return True

        # 4. Résolution du niveau requis
        niveau_requis = NiveauAcces.LECTURE if request.method in permissions.SAFE_METHODS else NiveauAcces.ECRITURE

        # Vérification d'une surcharge sur l'action spécifique (ex: @exige_niveau ou handler spécifique)
        handler = getattr(view, request.method.lower(), None)
        if handler and hasattr(handler, ATTR_NIVEAU):
            niveau_requis = getattr(handler, ATTR_NIVEAU)
        elif hasattr(view, ATTR_NIVEAU):
            niveau_requis = getattr(view, ATTR_NIVEAU)

        # 5. Contrôle d'accès par rapport au profil
        matrice = get_permissions_utilisateur_tenant(user, request)
        niveau_utilisateur = matrice.get(module, NiveauAcces.AUCUN)

        if niveau_utilisateur < niveau_requis:
            self.message = (
                f"Accès refusé au module '{module}'. "
                f"Niveau requis : {niveau_requis}, votre profil dispose de : {niveau_utilisateur}."
            )
            return False

        return True

    def has_object_permission(self, request, view, obj) -> bool:
        # Relayé vers la vérification de surcharge projet (Niveau 2) si applicable
        from apps.core.permissions import PermissionModule
        module = resoudre_module_requete(view, request)
        if not module:
            return True
        return PermissionModule.pour(module, 1)().has_object_permission(request, view, obj)
```

3.4 Étape 4 : Activation Globale dans REST_FRAMEWORK

Ouvrez config/settings/base.py et remplacez la liste par défaut des permissions pour inclure AutoModulePermission :

Fichier à éditer / créer : config/settings/base.py

```
# config/settings/base.py
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
        "apps.core.permissions_module.AutoModulePermission",
    ),
    # ... autres réglages DRF
}
```

SECTION 4 : SIGNAL D'ERREUR BAYÉSIEEN & PRE-MORTEM (KAHNEMAN / DEHAENE)

4.1 La Simulation Pre-Mortem (Daniel Kahneman)

Projetez-vous dans 6 mois : le système vient de passer en production. Une alerte critique survient : des utilisateurs se plaignent d'erreurs 403 inattendues, ou l'API s'effondre sous la charge. **Qu'est-ce qui a échoué ?** Voici les 5 pièges majeurs et leur parade architecturale immédiate :

Scénario de Défaillance	Conséquence Négative	Contre-Mesure Préventive Intégrée
1. Blocage de Swagger / OpenAPI	La documentation /api/v1/docs/ renvoie 403 Forbidden.	Exemption explicite dans PREFIXES_EXEMPTES pour /schema/ et /docs/.
2. Blocage du Endpoint /me/	L'utilisateur connecté ne peut plus charger son profil initial.	Exemption du chemin /api/v1/utilisateurs/me/ pour tout utilisateur authentifié.
3. Requêtes SQL N+1	Chaque sous-requête de vue réinterroge la table des permissions.	Attribut request._module_perms_cache mémorisant le dictionnaire en mémoire \$O(1)\$.
4. Route sans module détecté	Un endpoint générique échappe au contrôle ou est bloqué.	Si aucun module n'est identifié, la vue laisse passer vers les autres permissions déclarées.
5. Bypass involontaire d'actions sensibles	Une validation de bon passe avec le niveau Écriture (2) au lieu de Validation (3).	Décorateur @exige_niveau(NiveauAcces.VALIDATION) posé sur les actions critiques.

4.2 Le Signal d'Erreur Bayésien (Stanislas Dehaene - Pilier 3)

Neurosciences : Clarté Chirurgicale du Message d'Erreur

Selon la théorie du Cerveau Bayésien, notre cerveau progresse uniquement lorsqu'il reçoit un **signal d'erreur précis et non ambigu** (l'écart entre la prédiction et l'observation). Si votre test échoue avec un code 500 obscur, le cerveau ne progresse pas. En revanche, lorsque votre message de refus 403 formule exactement :

Accès refusé au module 'finance'. Niveau requis : 2 (Écriture), votre profil dispose de : 1 (Lecture).

L'administrateur ou le testeur comprend immédiatement la divergence et corrige la configuration sans friction.

SECTION 5 : CHECKLIST DE TESTS & VALIDATION AUTOMATISÉE

5.1 La Suite de Tests Unitaires & d'Intégration

Créez le fichier de test `apps/accounts/tests/test_auto_module_permissions.py`. Cette suite vérifie chaque branche de l'algorithme :

Fichier à éditer / créer : apps/accounts/tests/test_auto_module_permissions.py

```
# apps/accounts/tests/test_auto_module_permissions.py
import pytest
from django_tenants.utils import schema_context
from rest_framework import status
from rest_framework.response import Response
from rest_framework.views import APIView
from rest_framework.test import APIClient

from apps.accounts.models import Role, RoleModulePermission, Utilisateur
from apps.accounts.services.roles import initialiser_roles_par_defaut
from apps.core.enums import ModuleChoix, NiveauAcces, RoleGlobal, StatutUtilisateur
from apps.core.permissions_module import (
    AutoModulePermission,
    exempt_module_permission,
    exige_niveau,
)

SCHEMA = "demo"
HOTE = "demo.localhost"

# Vues de test simulant les modules
class DummyProjetView(APIView):
    module_name = ModuleChoix.PROJETS
    def get(self, request): return Response({"status": "ok_lecture"})
    def post(self, request): return Response({"status": "ok_ecriture"})

class DummyFinanceValidationView(APIView):
    module_name = ModuleChoix.FINANCE
    @exige_niveau(NiveauAcces.VALIDATION)
    def post(self, request): return Response({"status": "ok_validation"})

@exempt_module_permission
class DummyRouteExempteeView(APIView):
    def get(self, request): return Response({"status": "ok_exempt"})

@pytest.mark.django_db
def test_auto_module_permission_enforcement():
    with schema_context(SCHEMA):
        initialiser_roles_par_defaut()

        # Profil Magasinier : Stocks=Ecriture, Projets=Lecture, Finance=Aucun
        role_mag = Role.objects.get(code=RoleGlobal.MAGASINIER)
        RoleModulePermission.objects.filter(role=role_mag, module=ModuleChoix.PROJETS).update(niveau=NiveauAcces.LECTURE)
        RoleModulePermission.objects.filter(role=role_mag, module=ModuleChoix.FINANCE).update(niveau=NiveauAcces.AUCUN)

        mag_user, _ = Utilisateur.tous_objets.get_or_create(
            email="magasinier.test@demo.ci",
            defaults={"nom": "Koffi", "role_global": RoleGlobal.MAGASINIER, "statut": StatutUtilisateur.ACTIF}
        )

        client = APIClient()
        client.force_authenticate(user=mag_user)

        # 1. Lecture Projets -> Autorisé (200)
        req_get = client.get("/dummy-projets/", HTTP_HOST=HOTE)
        rep_get = DummyProjetView.as_view()(req_get.wsgi_request)
        assert rep_get.status_code == status.HTTP_200_OK

        # 2. Écriture Projets -> Refusé (403)
        req_post = client.post("/dummy-projets/", HTTP_HOST=HOTE)
        rep_post = DummyProjetView.as_view()(req_post.wsgi_request)
        assert rep_post.status_code == status.HTTP_403_FORBIDDEN

        # 3. Validation Finance -> Refusé (403)
        req_fin = client.post("/dummy-finance/", HTTP_HOST=HOTE)
        rep_fin = DummyFinanceValidationView.as_view()(req_fin.wsgi_request)
        assert rep_fin.status_code == status.HTTP_403_FORBIDDEN

        # 4. Route exemptée -> Autorisé (200)
        req_ex = client.get("/dummy-exempt/", HTTP_HOST=HOTE)
        rep_ex = DummyRouteExempteeView.as_view()(req_ex.wsgi_request)
        assert rep_ex.status_code == status.HTTP_200_OK
```

5.2 Commande d'Exécution & Taux de Couverture

Exécutez la suite de validation dans votre terminal de développement :

Fichier à éditer / créer : **Terminal PowerShell**

```
pytest apps/accounts/tests/test_auto_module_permissions.py -v
```

SECTION 6 : DÉFI HOMO DOCENS & CONSOLIDATION NOCTURNE

6.1 Le Défi Pédagogique « Homo Docens » (Lovett / Dehaene)

Vous n'avez véritablement maîtrisé une architecture logicielle que lorsque vous êtes capable de l'enseigner avec clarté sans aucune note sous les yeux. Voici votre épreuve d'élévation au niveau **Homo Docens** :

- **Défi 1 (Tableau Blanc - 3 minutes)** : Dessinez le flux d'une requête HTTP traversant `AutoModulePermission` et expliquez à un développeur junior pourquoi l'inspection de package Python (`view.__module__`) combinée à la mémorisation sur `request._module_perms_cache` élimine tout risque de surcharge de base de données.
- **Défi 2 (Arbitrage d'Équipe)** : Répondez à la question d'un collègue qui vous demande : « *Pourquoi ne pas avoir utilisé un `middleware Django` plutôt qu'une `permission DRF` ?* » (Réponse : Le `middleware` s'exécute avant le parsing DRF, ignore le système d'authentification JWT sans requêtes manuelles, ne connaît pas les `ViewSets` ni les décorateurs d'action DRF, et ne gère pas les formats de négociation de contenu pour les réponses d'erreur).
- **Défi 3 (Preuve par l'Exemple)** : Démontrez en direct dans le terminal comment une route non décorée d'un module interdit immédiatement un accès non autorisé sans qu'on ait eu à modifier son contrôleur.

6.2 Protocole de Consolidation Subconsciente Nocturne

Neurosciences : Activation de la Consolidation Nocturne

Le Replay Accéléré à 20x (Stanislas Dehaene - Pilier 4) :

Cette nuit, pendant vos phases de sommeil profond et paradoxal, votre hippocampe va rejouer les circuits synaptiques activés lors de la saisie de ce code à une vitesse 20 fois supérieure à l'éveil. Pour optimiser cette consolidation biologique :

1. Éteignez tout écran 30 minutes avant de dormir.
2. Dans l'état de somnolence (Rituel de Baudoin / Murphy), revivez la vision de votre API solide et sécurisée.
3. Laissez votre subconscient organiser la mémoire à long terme dans vos aires corticales permanentes.

Critère de Validation	Statut Attendu	Signature Développeur
Code tapé manuellement sans copier-coller	100% Manuel	[] Validé
Toutes les routes de modules protégées automatiquement	Secure by Default	[] Validé
Tests unitaires pytest au vert	100% Passed	[] Validé
Défi Homo Docens enseigné à un pair	Homo Docens	[] Validé